

Engineer Agent 原型 架构图与处理流程

五种设计模式完整实现（附真实运行轨迹）

配套项目：engineer_agent_prototype v0.1.0

覆盖模式：Pattern 1-5（Reflection / Plan and Solve / Tool Use / Multi-Agent / HITL）

2026 年 7 月 20 日

目录

- 1 原型概述
- 2 总体架构图
- 3 主循环处理流程（Pattern 2: Plan and Solve）
- 4 工具调用处理流程（Pattern 3: Tool Use）
- 5 反思、批判与协商机制（Pattern 1 / Pattern 4）
- 6 端到端执行时序（真实运行轨迹）

1 原型概述

本原型（`engineer_agent_prototype`）按"Engineer Large Model (Open Source)"架构图实现了一个可运行的工程领域多智能体系统，**完整落地全部五种设计模式**：P1 Reflection（generator-critic 自我批判环 + 教训沉淀）、P2 Plan and Solve（LangGraph 规划-执行-重规划主循环）、P3 Tool Use（MCP 风格受控工具层）、P4 Multi-Agent Collaboration（声明式专业代理 + `team.ask` 代理间协商）、P5 Human-in-the-Loop（权限引擎 + 高危拦截 + 验收门禁）。系统以"户外储能电源设计"为示例任务，完整跑通"任务分解 → 专业代理执行 → 检查失败 → 反思 → 协商 → 调参 → 复检 → 批判修订 → 生成报告 → 高危拦截 → 验收"的全流程，离线无需 API Key 即可复现，也可一键切换真实大模型。原型的五模式框架与具体任务解耦：不换一行代码即以"智能电表"新任务实测跑通（真实模型自主规划 5 步全部通过），任务相关性只存在于声明式代理定义与工具模拟数据，可按需替换。

表1 技术栈与参考项目

层	选型	参考项目	对应模式
规划执行主循环	LangGraph Plan-and-Execute	langchain-ai/langgraph	P2
计划优化思想	DAG 并行 / 重规划（架构借鉴）	SqueezeAILab/LLMCompiler	P2
工具接入层	MCP 风格受控调用层（最小实现）	modelcontextprotocol/servers	P3
自我批判环	generator-critic 双节点 + 教训沉淀	LangGraph Reflection 教程 / Reflexion / Self-Refine / CRITIC	P1
代理间协商	<code>team.ask</code> 元工具 + 声明式专业代理	crewAI / AutoGen / sst/opencode	P4
权限模型	<code>allow / ask / deny</code> + 通配符 + 验收门禁	sst/opencode / LangGraph interrupt / AG-UI	P5
LLM 抽象	MockLLM（离线） / OpenAI 兼容接口	—（可接 vLLM/Ollama 开源基座）	—

2 总体架构图

原型采用五层结构：接入层（CLI）→ 主循环层（LangGraph 状态图）→ 代理层（声明式专业 Engineer）→ 受控工具层（MCP 风格）→ 工程环境层。所有工具调用必须穿过受控工具层；反思模块横跨执行与重规划两个环节，如图1所示。

2 总体架构图

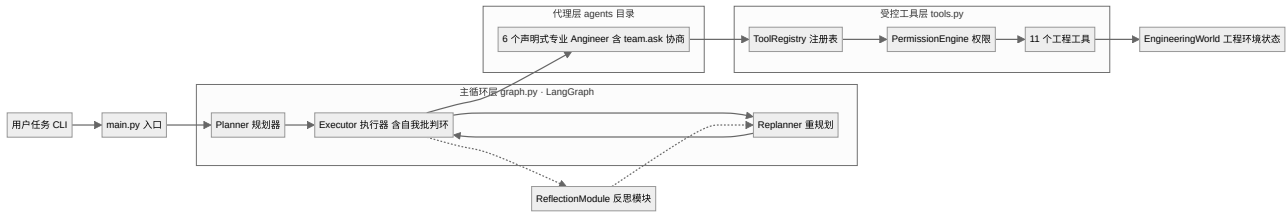


图 1 原型总体架构图（五层结构，虚线为反思数据流）

各层与原始架构图的对应关系如表 2 所示——架构图上的每个功能框都能在代码中找到落地位置。

表 2 原始架构图模块 → 原型代码映射

架构图模块	代码位置	落地方式
Task Decomposition	graph.py planner_node	LLM 将任务分解为 {代理, 指令} 步骤列表
Actions / 底部专业 Engineer 总线	graph.py executor_node + agents/*.md	每步路由到对应专业代理，驱动 ReAct 小循环
Decisions	replanner 条件边	继续推进 / 重规划 / 结束 三向分支
Space Control	MAX_STEP_ITERS=8 / MAX_REPLANS=2 / MAX_CRITIQUES=2	单步迭代熔断 + 重规划上限 + 批判轮次上限，防止失控循环
Reflection（自我批判）	graph.py executor 批判环 + llm.critique()	final 先经 critic 评审，revise 则带意见重写（最多 2 轮）
Multi-Agent 协作	graph.py _consult + team.ask 元工具	代理可就专业问题咨询其他代理，禁嵌套、3 步上限、全程留痕
HITL 验收门禁	replanner done 分支 final_acceptance	全部步骤完成后须人工验收方可结束流程
Engineer Tools	tools.py ToolRegistry	声明式 schema + 权限包裹 + 日志的受控调用层
Engineer Knowledge Graph	kg_query 工具	工程知识图谱查询（模拟，只读）
Data Preparation and Transform	data_transform 工具	传感器/CAD 数据清洗变换（模拟）
Debug and Refine	reflection.py + replanner_node	执行期自省 + 计划级修正
Optimization	optimization.set_param 工具	修改设计变量（ask 权限）
Writing Report / Apply Results	report.write / production.apply	写报告需确认；应用生产默认 deny

架构图底部绿色总线的对照关系：Structure Design / Chip Design / PCB CAM / Safety / Discharge Monitoring 均有独立代理文件；**Inspection Engineer** 的职责落在 `inspection.query` 工具上（由 Discharge Monitoring 代理持有调用）；**Industry Engineer** 与 "Etc..." 暂未实现——往 `agents/` 目录加一个 `md` 文件即完成扩展，无需改动主循环代码，这正是架构图为扩展预留的方式。

```
engineer_agent_prototype/  
├── main.py                # CLI 入口 (--auto-approve / --real / --task)  
├── agents/                # 6 个声明式专业代理 (一个 md 文件 = 一个 Engineer)  
│   ├── structure_design.md  chip_design.md  pcb_cam.md  
│   ├── safety.md           discharge_monitoring.md  chief.md  
├── engineer/  
│   ├── graph.py           # LangGraph Plan-Execute-Replan 主图 (P2 核心)  
│   ├── tools.py           # MCP 风格工具注册表 + 受控调用层 (P3 核心)  
│   ├── permissions.py     # allow/ask/deny 权限引擎 (P5)  
│   ├── reflection.py      # 周期自检 + 教训沉淀 (P1)  
│   ├── agents_loader.py   # 声明式代理加载器  
│   └── llm.py             # LLM 抽象: MockLLM / OpenAILLM  
└── docs/technical-selection_CN.md # 选型、参考项目与原因 (中英双语)
```

3 主循环处理流程 (Pattern 2: Plan and Solve)

主循环是 LangGraph 编译的状态图，含三个节点与一条条件边，对应 LangChain 官方 Plan-and-Execute 范式：**Planner** 产出显式计划（步骤列表），**Executor** 逐步调派专业代理执行，**Replanner** 在每一步之后决策"推进 / 修正 / 结束"，如图 2 所示。计划是显式数据结构，失败步骤会被重新入队——这借鉴了 LLMCompiler "计划可修正" 的思路。

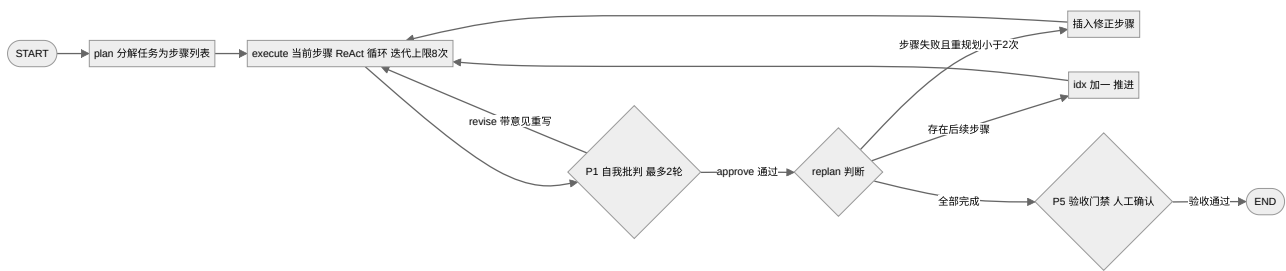


图 2 主循环状态图 (Planner → Executor → Replanner, 含 P1 批判环、P5 验收门禁与 Space Control 熔断)

表 3 主循环节点职责

节点	输入 → 输出	职责说明
----	---------	------

pl	任务文本 →	调用 <code>llm.decompose()</code> ：MockLLM 按关键词路由到专业代理；真实模型按代理
an	plan 步骤列表	roster 生成 JSON 计划；末尾自动追加总工（chief）汇总步骤
ex	当前步骤 →	加载步骤指定的代理（系统提示词 + 授权工具清单 + 权限覆盖），进入 ReAct 小循环：
ec	执行结果	<code>llm.act()</code> 决策 → 工具受控调用 → 结果写入 scratchpad；代理给出 final 后先经
ut		<code>llm.critique()</code> 自我批判，revise 则带批判意见重写（最多 2 轮），approve 方为
e		本步定稿；每步指令自动注入最近 4 条前置步骤结果摘要作为上下文
re	最近结果 +	步骤失败且重规划次数未达上限时，把携带最近一条教训的修正步骤插回队列；否则推
pl	lessons →	进下一步；全部完成则触发 P5 验收门禁（ <code>final_acceptance</code> ，须人工确认）后结
an	路由决策	束

Space Control（防失控设计）。 对应架构图的 Space Control 框，原型设三道硬约束：单步最多 `MAX_STEP_ITERS=8` 次工具调用（防止代理在一步内空转），全程最多 `MAX_REPLANS=2` 次重规划（防止"失败—重试"无限循环），单步最多 `MAX_CRITIQUES=2` 轮自我批判（防止"批判—修订"无限循环）。注入"永不收敛"故障的单元测试已验证：系统在 3 次执行、2 次重规划后正常终止，无死循环。

4 工具调用处理流程 (Pattern 3: Tool Use)

代理不直接接触任何外部系统，所有调用穿过受控调用层 `ToolRegistry.invoke()`，固定执行"schema 校验 → 权限匹配 → 执行 → 日志 → 反思"五段管线，如图 3 所示。这正是文章对 Tool Use 的定义：模型只能通过声明了用途、参数与输出格式的受控接口工作，权限收敛在工具级而非放给模型。

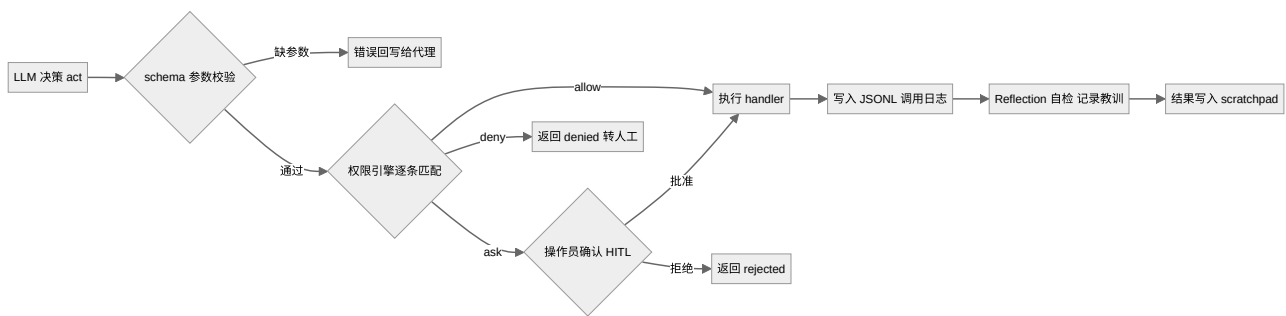


图 3 单次工具调用的受控管线（含 HITL 三支）

权限引擎参考 `opencode`：规则按通配符逐条匹配、**后匹配优先**，代理文件中的权限覆盖追加在全局规则之后（代理级优先）。全局默认规则如表 4 所示。

表 4 全局权限规则（`permissions.py`）

工具模式	决策	理由
------	----	----

kg_query / inspection.query / discharge_sensor.*	allow	只读查询：知识图谱、检测记录、放电监测传感器
team.ask	allow	代理间协商元工具：不触外部系统，仅驱动被咨询方的一次只读式回答
data_transform / structure_calc.* / cam_check.* / chip_sim.*	allow	纯内部计算与仿真，无副作用
optimization.*	ask	修改设计参数，需工程师确认
safety_audit.* / report.write	ask	安全审计结论与落盘文件，需确认
production.*	deny	应用到生产为高危操作，原型默认禁止，只能人工改规则放行

表5 工具清单（11 个工程工具 + 1 个代理间协商元工具，MCP 风格 schema 声明）

工具	用途	只读	授权代理
kg_query	工程知识图谱查询（材料/工艺/器件）	是	structure_design, chip_design, pcb_cam
data_transform	数据清洗与变换	否	safety
structure_calc.run	结构强度校核	否	structure_design
chip_sim.run	芯片功耗/时序/DRC 仿真	否	chip_design
cam_check.run	PCB 载流/线宽 CAM 检查	否	pcb_cam
optimization.set_param	优化设计参数	否	pcb_cam
safety_audit.run	安规/温升/防护审计	否	safety
inspection.query	质量检测记录查询	是	discharge_monitoring
discharge_sensor.read	放电监测数据读取	是	discharge_monitoring
report.write	生成工程报告	否	chief
production.apply	变更应用到生产（高危）	否	chief（且被 deny 拦截）
team.ask（元工具）	代理间协商：向另一专业代理提问并带回结构化答复	是	全部 6 个代理（禁嵌套，3 步上限）

5 反思、批判与协商机制 (Pattern 1 / Pattern 4)

架构图上的 Debug and Refine 框在原型中拆成三层落地：**结果级自我批判**由 Executor 内的 generator-critic 环完成（参考 LangGraph Basic Reflection 教程的双节点结构，思想源自 Reflexion / Self-Refine / CRITIC）；**执行期自省**由 ReflectionModule 完成（参考 hermes-agent 的内核自检循环）；**计划级修正**由 Replanner 节点完成。三者通过共享的 lessons 列表与 scratchpad 衔接：批判意见写回 scratchpad 驱动修订，执行期沉淀的教训则是重规划时修正步骤的输入。

5.1 自我批判环 (P1 核心)

代理给出 final 后不直接定稿，而是调用 `llm.critique()` 由同一 LLM 切换为评审者视角，对照任务指令审查结论的完整性与证据充分性，返回 `approve` 或 `revise + 批判意见`。`revise` 时批判意见写入 scratchpad，代理带着意见重新生成结论，最多 `MAX_CRITIQUES=2` 轮。k3 实测中该机制多次发挥实质作用：抓到结构代理“安全系数 1.37 与 1.52 前后矛盾”、批判 Safety 代理“证据不足，暂不予通过”、指出总工首版报告“缺少各专业结果的结构化汇总、风险分级与变更去向”并促成修订版。

5.2 教训沉淀与周期自检

ReflectionModule 在每次工具调用后按规则触发，把异常与经验固化为结构化教训，供同步续跑与重规划复用。触发规则如表 6 所示。

表 6 ReflectionModule 触发规则 (reflection.py)

触发条件	动作	示例 (来自真实运行)
工具结果 <code>status = fail</code>	沉淀一条结构化教训	"工具 <code>cam_check.run</code> 失败：线宽 0.20mm 小于要求 0.30mm……→ 调整策略后重试"
<code>status = error</code> (<code>schema</code> 校验未过)	记录"按工具 <code>schema</code> 修正参数"	"工具 <code>structure_calc.run</code> 调用错误：缺少参数 ['model'] → 按工具 <code>schema</code> 修正参数"
<code>status = denied / rejected</code>	记录"此类操作需人工审批"	"工具 <code>production.apply</code> 被人工/策略拦截：此类操作需走人工审批"
每 3 次工具调用	周期性自检，评估轨迹是否正常	"周期性自检 (第 3 次工具调用)：轨迹正常，继续执行"

5.3 代理间协商 (P4 核心)

多智能体协作不止于“任务分派”：每个代理还持有 `team.ask` 元工具，可就跨界专业问题咨询其他代理。协商由主图统一调度（被咨询方走独立 3 步小循环，禁止嵌套 `team.ask`、同样受权限引擎约束、调用留痕进 JSONL 日志），避免 CrewAI 式自由对话的失控风险，更接近 AutoGen 的受

控消息传递与 opencode 的声明式子代理调用。k3 实测中模型曾自发咨询不存在的 "certification_engineer"，结构化错误回写 + 教训沉淀 + 重规划三件套稳稳接住，代理随后改为咨询真实存在的 Safety 代理——这恰好验证了协商机制的容错设计。

以示例任务中的 PCB CAM 步骤为例，完整闭环为：cam_check.run (0.20mm) → fail → 反思记录教训 → team.ask 咨询 Safety 代理（P4 协商：确认整改方向不引入安全风险）→ 代理决策调参 optimization.set_param (0.35mm, ask 权限, 操作员批准) → 复检 cam_check.run (0.35mm) → 通过 → P1 自我批判评审后定稿。注意工具之间存在真实状态依赖：set_param 修改了 EngineeringWorld 中的线宽，复检结果因此改变——这让"检查—协商—优化—复检"闭环是可复现的真实逻辑，而非写死的表演脚本。

6 端到端执行时序（真实运行轨迹）

图 4 给出示例任务中最关键的 PCB CAM 步骤的时序（含 P4 协商、HITL 确认、P1 批判与 P5 验收门禁），表 7 则是一次完整真实运行（python main.py --auto-approve）的全部 12 次工具调用轨迹。

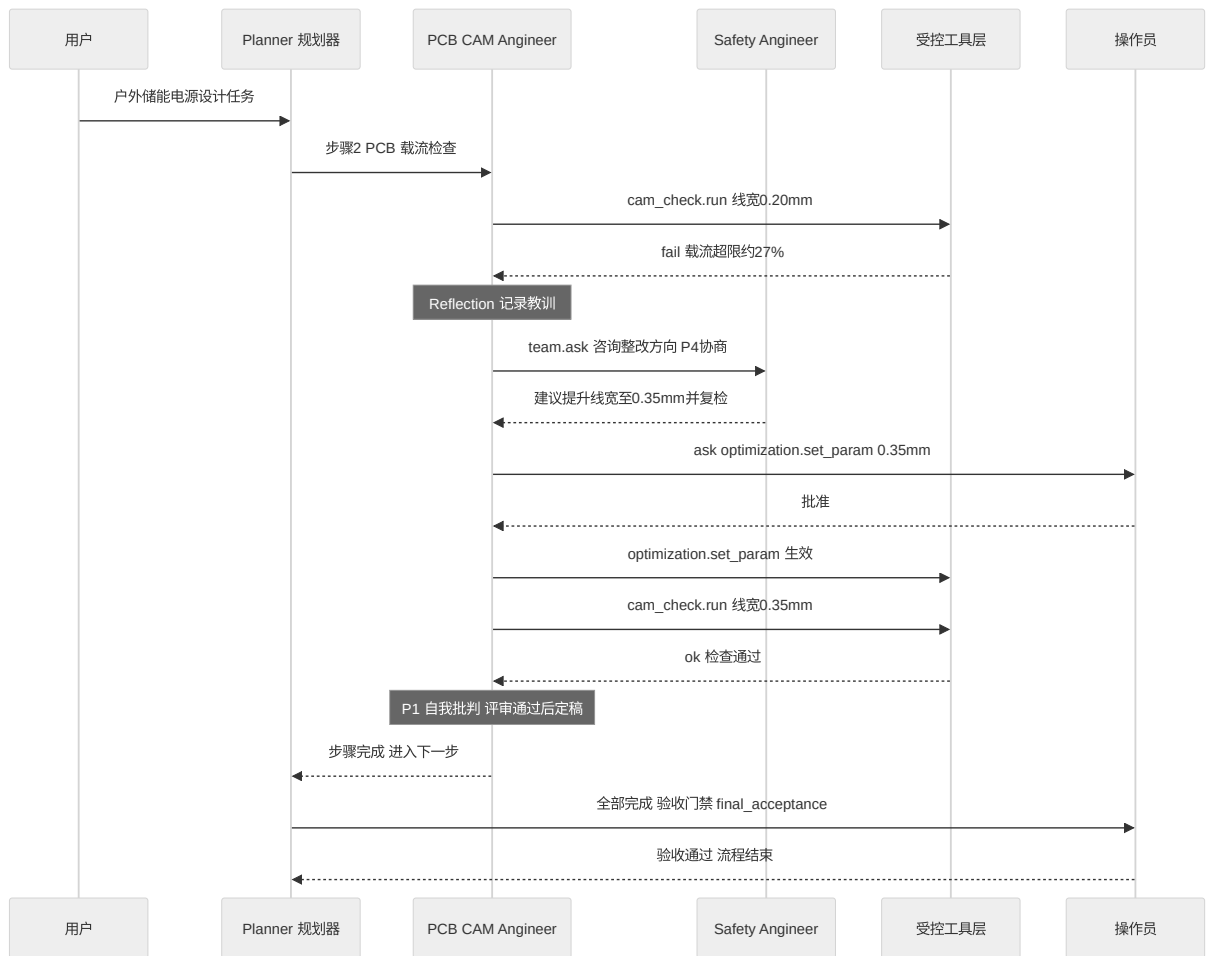


图 4 PCB CAM 步骤时序图（检查失败 → 反思 → P4 协商 → HITL 调参 → 复检 → P1 批判定稿 → P5 验收）

表 7 户外储能电源示例任务完整执行轨迹（真实运行，共 12 次工具调用，含 1 次代理间协商）

#	步骤（代理）	工具调用	权限	结果
1	结构校核（structure_design）	kg_query 铝合金6061	allow	命中材料参数
2		structure_calc.run 电池仓主梁	allow	应力 182MPa，安全系数 1.37，通过
3	CAM 检查（pcb_cam）	cam_check.run 0.20mm	allow	fail 载流超限约 27% → 反思记教训
4		team.ask 咨询 safety	allow	P4 协商 ：建议线宽提至 0.35mm 并复检温升
5		optimization.set_param 0.35mm	ask→ 批准	设计参数已更新
6		cam_check.run 0.35mm	allow	通过 → P1 批判评审后定稿
7	安全审计（safety）	data_transform 温升监测数据	allow	清洗 12480 行
8		safety_audit.run 整机安规	ask→ 批准	通过，1 项整改建议
9	放电监测 （discharge_monitoring）	discharge_sensor.read pack-01	allow	SOC 62%，41.5°C，正常
10		inspection.query 成品	allow	一次合格率 98.6%
11	汇总（chief）	report.write engineering_report.md	ask→ 批准	报告已落盘（初版经 P1 批判后修订）
12		production.apply 线宽变更	deny	被权限策略拦截，转人工审批

运行结束产生两份可追溯产物：output/engineering_report.md（自动汇总各专业执行结果与教训记录）与 output/tool_calls.jsonl（12 条带时间戳的调用日志，含 team.ask 协商留痕）。整条轨迹完整覆盖全部五个设计模式：P1（自我批判环 + 教训沉淀）、P2（规划-执行-重规划主循环）、P3（受控工具调用）、P4（任务分派 + team.ask 代理间协商）、P5（ask 确认、deny 拦截与验收门禁）。